ДЗ лекции 3. Глубокое обучение и нейронные сети

Дисциплина: Методы искусственного интеллекта I
Выполнил: Макеев Глеб Борисович, группа BHEBDPS-24-MII-5
Тема: письменные ответы на вопросы лекции 3 (слайд 16) + подготовка к лекции 4
Дата: 18.10.2026

---

1. Почему Трансформеры «убили» RNN?

Коротко: RNN обрабатывает последовательность *последовательно*, а Attention — *сразу всю*.

Критерий	RNN / LSTM	Transformer
Параллелизм по длине	нет: `h_t` зависит от `h_{t-1}`, вычисление строго по шагам	да: все позиции считаются одновременно (матричные умножения)
Путь градиента между токенами	O(n) — ошибка «затухает» по пути	O(1) — Attention связывает любые две позиции напрямую
Утилизация GPU	низкая: узкое место — рекуррентность, а не математика	высокая: чистые `matmul` / `softmax` — идеальные операции для тензорных ядер
Масштабирование (data + params)	быстро упирается в потолок качества	масштабируется до миллиардов параметров (BERT, GPT)
Память на шаг	O(1) (один скрытый вектор)	inference требует KV-cache, растущий с длиной контекста

Дополнительно: Attention оказался *универсальнее*. Одна и та же архитектура работает в NLP (перевод, GPT), в зрении (ViT), в аудио (Whisper), в биологии (AlphaFold), тогда как RNN — это архитектура «специально под последовательности». Отсюда историческая смена: 2014–2017 — LSTM/GRU + seq2seq; 2017 «Attention Is All You Need» → 2018+ — Transformer везде.

Важная оговорка. «Убили» — это про NLP и про качество на больших данных. Слабое место Transformer — квадратичная сложность по длине $O(n^2)$; поэтому живы линейные аппроксимации внимания и state-space модели (Mamba/S4), а в нишах, где рекуррентность — достоинство, LSTM/GRU никуда не ушли (см. вопрос 2).

---

2. Почему RNN/LSTM критичны для временных рядов и Edge-устройств?

Edge-устройства (микроконтроллеры, телефоны, камеры, датчики, embedded NPU):

1. **Размер.** LSTM/GRU — единицы тысяч параметров (десятки КБ), помещается в SRAM

микроконтроллера; Transformer с attention — сотни МБ и обязательный KV-cache.

1. **Память и режим потоковой работы.** LSTM хранит *один* скрытый вектор и обновляет

его за O(1) на каждый новый отсчёт — естественный streaming/online-инференс. У Transformer память растёт линейно с длиной контекста.

1. **Энергия и latency.** Один шаг рекуррентности = пара матрично-векторных умножений:

миллисекунды и милливатты, без пересчёта всей последовательности.

1. **Готовый тулчейн.** `TensorFlow Lite Micro`, `CMSIS-NN`, `ONNX Runtime Mobile`,

аппаратные NPU/микроконтроллеры отлично оптимизируют именно рекуррентные ячейки.

Временные ряды (промышленность, финансы, энергетика, IoT):

1. **Индуктивный bias под последовательность:** память и ворота «забудь/запомни»

заложены в архитектуру — это адекватная априорная модель для данных со временем, особенно на **малых** выборках, где Transformer переобучается.

1. **Онлайн-прогноз.** Предиктивное обслуживание и мониторинг требуют предсказания

«на лету», по одному измерению: LSTM это умеет по определению.

1. **Нерегулярные и шумные данные.** Пропуски датчиков, разная частота — LSTM хорошо

переносит, тогда как attention чувствителен к позиционному кодированию.

1. **Практика.** В задачах прогноза часто побеждают не «чистые» модели, а гибриды и

специализированные решения: LSTM/GRU + CNN (ConvLSTM), TCN, N-BEATS, Gradient Boosting на лагах, а из нового — Mamba/SSM. В embedded и на производстве классическая LSTM до сих пор стандарт де-факто.

---

3. Подготовка к лекции 4: механизм Attention (краткий конспект)

- **Идея.** Для каждого токена (`Query`) ищем, на какие позиции (`Keys`) смотреть, и

берём взвешенную сумму их представлений (`Values`) — «мягкий поиск по базе памяти».

- **Формула:**

```
Attention(Q, K, V) = softmax(Q·Kᵀ / √d_k) · V
```

Деление на `√d_k` нужно, чтобы скалярные произведения не «разъезжались» по модулю и softmax не превращался в one-hot (градиенты не пропадали).

- **Self-attention.** Q, K, V получаются из одного и того же входа линейными

проекциями — токен «смотрит» на все остальные токены последовательности.

- **Multi-head.** Несколько независимых голов внимания (разные «точки зрения»:

синтаксис, семантика, дальние связи), затем конкатенация проекций.

- **Позиционность.** В отличие от RNN, attention ничего не знает о порядке, поэтому

порядок добавляют позиционными кодировками (синусы/косинусы или обучаемые векторы), в современных моделях — RoPE.

- **Блок трансформера.** Multi-head attention → Add & LayerNorm → Feed-Forward

(MLP, GeLU) → Add & LayerNorm; стек из N таких блоков.

- **Цена.** Память и время $O(n^2)$ по длине последовательности — главный объект

оптимизации (FlashAttention, sparse/linear attention).

Связь с лекцией 3: Attention — это та же «нейронная сеть + градиентный спуск + backpropagation», но вместо рекуррентности он строит обучаемую матрицу связей между всеми элементами последовательности сразу, убирая узкое место RNN.